

directory paths, email addresses, user names and other bits of information.

Any such information can be stored in variables which are embedded in configuration files. This way you can change a parameter in one place, and have it automatically update all the configuration files that need that value.

To manipulate text configuration files, Ansible include a rather powerful template engine called Jinja2. Using Jinja2 you can perform advanced text manipulation, such as looping over variables to build lists, and applying filters to text content. There is a simple Ansible command that can process a configuration file template, and copy it in the right place on the remote server.

Facts

When you run an Ansible playbook it first gathers a large collection of facts about the remote server such as the server architecture, OS, kernel details, date and time, hardware details, network connection details (like the IP address), hard disk partition details, and a lot more. In addition to using the variable you supply yourself, you can also use these gathered facts in your configuration files.

For instance, to set up the load-balancer situation we described before, the load-balancing servers' config file could automatically fill in the IPs of all your app servers. Now you could just increment a variable that controls the number of app servers and Ansible will automatically set up the new app server and include it in the load-balancer config.

Building the Linux Kernel with Ansible

Lets now create an Ansible playbook that will compile the Linux kernel for us. Before we can begin writing our script, we should understand the manual steps involved in this process. Our script should perform the following steps:

- Install a compiler and other developer tools to compile the kernel. Remember we assume it is a fresh system, Ansible will only install these packages if not already present.
- Install `git` to get a local copy of the Linux source code.
- Run `make mrproper` in the source directory to clean it.
- Run `make defconfig` to create a default configuration file. Better yet, we could build a nice config and copy it to the server instead.
- Run `make -j4`. Here 4 specifies how many compilation steps to run in parallel. Increase if your system can handle more.
- Run `make tarxz-pkg` make a compressed package with the kernel. Or you could run `make rpm-pkg` or `make deb-pkg` to create an `rpm` or `deb` package.

*pointers

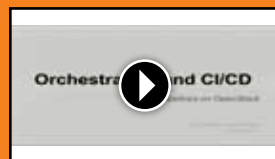
>>Interesting developer news



*Ansible Quick Start Video

>>This official video by Ansible will help you get better acquainted with the tool.

dgit.in/AnsibleQSVideo



*CI/CD with Ansible

>>If you want a quick look into how to achieve continuous integration and delivery with Ansible this video does just that.

dgit.in/AnsibleCDCI



*Etymology of Jargon

>>Where does some of the weirder or confusing terms and conventions in programming come from? Watch this video:

dgit.in/ProgJargon



*SaltStack

>>The four major players in the configuration management space are contrasted and compared in this Google Hangout video by Rackspace.

dgit.in/AnsibleHangout



If you have an account on Amazon AWS or similar, you could create a temporary server to run all this on. Else, it is possible to test with local commands on your computer as well. We will assume you are running Ansible commands locally in the following steps as that will work for most people. You could also create a virtual machine and manage that using Ansible.

Our Ansible Playbook

Lets start by creating our Ansible playbook script and call it `build-kernel.yaml` with the following contents:

```
---
- hosts: localhost
  vars:
    code_path: /home/ubuntu/linux/
  tasks:
    - name: Update System
      become: yes
      apt: upgrade=yes update_cache=yes
...
```

Here the first line `---` and the last line `...` are conventions of the YAML format, and not specific to Ansible. YAML is a text format for representing data like XML or JSON, but we won't delve much more into it here. The Wikipedia page on YAML is a good introduction to the format.

What we see here, is a playbook with one task. This task has a name of `Update System`; this is what will be shown in the terminal when this playbook is run. The actual task here is called `apt`, which is the name of the package manager on Debian, and Debian-derived versions of Linux like Ubuntu and Mint. The `become: yes` part of this task is to ensure that this command runs as the superuser.

If youre not using a Debian-based distro, you should probably omit the above task, and replace `apt` with `package` which works on most distros. There are also support for distro/language/framework-specific package managers like `yum`, `zypper`, `pip`, `gem`, `npm` and more.

What we also see is the declaration of one variable, the code path. This is where we will keep the source code. This allows us to write this path once, and reuse it wherever else needed.

We can now quickly add two more tasks that install the build tools and git:

```
- name: Install build tools
  become: yes
  apt: name=build-essential state=present
- name: Install git
  become: yes
  apt: name=git state=present
```

Here, we are providing two parameters, a package name, and a state. This will result in